

Evaluación Experimental y Análisis Crítico de una Estructura Aumentada para Reportes RAP

Autor: Maykoll Vanegas Silva | Docente: Ana Tapia, Ph.D. | Materia: Análisis de Algoritmos

Resumen— Este informe presenta la evaluación experimental de una estructura de datos aumentada diseñada para almacenar y consultar informes de retroalimentación automática de presentaciones orales del sistema RAP (Ochoa et al., 2018). Frente a esquemas tradicionales basados en barridos secuenciales $O(N)$, se implementó una arquitectura desacoplada en dos etapas: (i) índices secundarios en memoria mediante cuatro árboles AVL aumentados con invariantes (C , S , SS) que resuelven consultas estadísticas por intervalo con costo $O(\log D_m)$ en el peor caso, y (ii) almacenamiento primario persistente en un motor LSM-Tree (LevelDB 1.23) con Bloom Filter nativo y una implementación personalizada de Ribbon integrada mediante la interfaz extensible *FilterPolicy*. Las pruebas sobre 1,000, 10,000 y 50,000 reportes demuestran factores de aceleración de hasta **9,300x** en agregaciones de rango y reducciones de latencia de hasta el 18.2% en lecturas puntuales según el estado de la caché, analizando de forma crítica las fortalezas asintóticas, el efecto de la discretización de claves y los compromisos de concurrencia y memoria.

1. Introducción y Contexto Operativo de RAP

El sistema RAP (*Automatic Presentation Feedback*, Ochoa et al., 2018) provee retroalimentación sobre habilidades de exposición oral mediante sensores multimodales de bajo costo. Cada presentación genera un informe que resume métricas cuantitativas de contacto visual, volumen y postura. En su concepción original, RAP almacena reportes como archivos independientes o registros relacionales sin indexación secundaria multidimensional. Al acumular miles de presentaciones a lo largo de diversos periodos académicos, responder consultas agregadas de cohorte o clasificaciones mediante barridos lineales (**Fuerza Bruta**) impone una complejidad $O(N)$ que compromete la respuesta en tiempo real.

2. Modelo de Informes y Arquitectura Algorítmica

Cada informe se modela formalmente como $r_i = (id_i, u_i, t_i, M_i, D_i)$, donde id_i es el identificador único, u_i el expositor, t_i la marca de tiempo, D_i el payload JSON (~1 KB con desglose multimodal detallado) y $M_i = (g, z, v, p)$ reúne cuatro métricas continuas dentro de los dominios definidos para la evaluación: puntaje global $g \in [1.0, 5.0]$ y componentes normalizados de contacto visual $z \in [0.0, 1.0]$, volumen $v \in [0.0, 1.0]$ y postura $p \in [0.0, 1.0]$, acordes a las escalas observadas en la literatura de RAP. Se implementó una **arquitectura desacoplada en dos fases**:

• **Fase 1 (Índices Secundarios Aumentados en RAM)**: Cuatro árboles AVL independientes (I_g, I_z, I_v, I_p) . Cada nodo almacena la clave métrica k_x , una lista de identificadores L_x y campos aumentados de subárbol: cardinalidad $C(x) = C(x_L) + |L_x| + C(x_R)$, suma $S(x) = S(x_L) + k_x |L_x| + S(x_R)$ y suma de cuadrados $SS(x) = SS(x_L) + k_x^2 |L_x| + SS(x_R)$, actualizados en $O(1)$ durante inserciones y rotaciones. Mediante la función de prefijo AgregadoLE(t), las estadísticas de cualquier intervalo $[a, b]$ se calculan como $A_{[a,b]} = A_{\leq b} - A_{< a}$ con costo $O(\log D_m)$ en el peor caso, independiente del número de elementos contenidos en el rango.

• **Fase 2 (Almacenamiento Persistente en LSM-Tree)**: Los payloads D_i residen en LevelDB 1.23. Tras resolver los predicados en RAM y determinar el conjunto de identificadores R_Q mediante intersección ordenada por cardinalidad creciente, se implementó $\text{MultiGet}(R_Q)$ a nivel de la aplicación como una operación por lotes sobre llamadas individuales $\text{Get}()$ de LevelDB, asistidas por filtros probabilísticos (Bloom nativo y Ribbon personalizado mediante *FilterPolicy*).

3. Metodología Experimental y Validación de Correctitud

Los experimentos se ejecutaron sobre Linux 6.8 (Intel Core i5-12600KF, 32 GB RAM, NVMe) evaluando tres escalas: **$N = 1,000$** , **$N = 10,000$** y **$N = 50,000$** informes sintéticos generados dentro de los dominios definidos (semilla fija $\text{seed}=42$). Las mediciones corresponden a microbenchmarks con datos calientes en caché L1/L2 ejecutados 50 veces consecutivas para promediar latencias; en este régimen sub-microsegundo los tiempos reportados reflejan el orden de magnitud (decenas de nanosegundos) sujeto al overhead de medición del temporizador, predicción de saltos y optimizaciones del compilador.

Validación de Correctitud: Mediante pruebas unitarias automáticas (`test_correctness.cpp`) se verificó que el balance $|h_L - h_R| \leq 1$ se preserva rigurosamente (altura máxima $h = 11$ con 401 claves discretas), y que la media, varianza y desviación estándar coinciden con la implementación de Fuerza Bruta dentro de una tolerancia numérica de 10^{-6} .

4. Resultados Experimentales Consolidados

A continuación se sintetizan las métricas de rendimiento obtenidas en las tres escalas evaluadas.

Tabla 1: Tiempo de Ingesta e Indexación en Memoria

Escala N	Ingesta Fuerza Bruta (std::vector)	Ingesta + 4 AVL Aumentados	Latencia por Informe
N = 1,000	0.354 ms	0.282 ms	0.282 μ s / informe
N = 10,000	3.277 ms	2.590 ms	0.259 μ s / informe
N = 50,000	15.585 ms	11.351 ms	0.227 μ s / informe

Nota metodológica sobre Tabla 1: La ingesta en std::vector realiza la copia profunda del objeto RAPReport completo (incluyendo el payload JSON de ~1 KB y reasignaciones dinámicas de memoria para cada informe). En contraste, la arquitectura propuesta desacopla el payload hacia LevelDB y únicamente indexa en memoria las claves numéricas y punteros/IDs ligeros en los cuatro AVL (con $D_m \leq 401$ claves discretas, el número de nodos es acotado y no se clonan los payloads), explicando la menor latencia de indexación en RAM frente a la clonación en vector.

Tabla 2: Consultas Estadísticas de Rango (Algoritmo 4: AgregadoLE)

Escala N	Intervalo Evaluado	k Items	AVL Aumentado $O(\log D)$	Fuerza Bruta $O(N)$	Factor Speedup
N = 1,000	Puntaje Global [3.50, 3.80]	358	0.0253 μ s (25 ns)	3.55 μ s	140.3x
N = 10,000	Puntaje Global [3.50, 3.80]	3,570	0.0299 μ s (30 ns)	39.84 μ s	1,331.7x
N = 50,000	Puntaje Global [3.50, 3.80]	17,896	0.0273 μ s (27 ns)	253.54 μ s	9,300.9x
N = 50,000	Mirada Audiencia [0.60, 0.85]	28,074	0.0391 μ s (39 ns)	220.13 μ s	5,629.8x
N = 50,000	Volumen de Voz [0.65, 0.90]	31,590	0.0334 μ s (33 ns)	182.15 μ s	5,453.6x

Tabla 3: Selección Top-k de Expositores (Algoritmo 7: TopK)

Escala N	Consulta	AVL Top-k $O(\log D + k)$	Fuerza Bruta $O(N \log k)$	Factor Speedup
N = 1,000	Top-10 Mejores Expositores	0.133 μ s	6.09 μ s	45.8x
N = 10,000	Top-10 Mejores Expositores	0.130 μ s	57.60 μ s	441.5x
N = 10,000	Top-50 Mejores Expositores	0.411 μ s	64.43 μ s	156.8x
N = 50,000	Top-10 Mejores Expositores	0.167 μ s	313.77 μ s	1,884.1x
N = 50,000	Top-100 Mejores Expositores	0.623 μ s	355.14 μ s	570.0x

Tabla 4: Materialización en Almacenamiento Persistente (LevelDB 1.23 + Filtros AMQ)

Escala N	Claves RQ	Sin Filtro (Baseline)	Bloom Filter (10 bpk)	Ribbon Filter (10 bpk equiv)	Ahorro AMQ
N = 1,000	215 items	3.298 μ s / op	2.712 μ s / op	2.698 μ s / op	18.2% ahorro
N = 10,000	2,146 items	3.271 μ s / op	2.687 μ s / op	2.675 μ s / op	18.2% ahorro
N = 50,000	10,801 items	2.282 μ s / op	2.292 μ s / op	2.324 μ s / op	Caché caliente
Misses (1k)	1,000 claves	0.057 μ s / op	0.066 μ s / op	0.049 μ s / op	Descarte veloz

5. Análisis Crítico: Fortalezas de la Propuesta

- 1. Invarianza Temporal Asintótica:** La mayor fortaleza es la estabilidad de AgregadoLE con costo $O(\log D_m)$ en el peor caso. Las mediciones de microbenchmark muestran tiempos del orden de ~25 a 30 nanosegundos tanto para $N = 1,000$ como para $N = 50,000$, incluso cuando el intervalo engloba más de 17,800 informes. Al eliminar la dependencia de la cantidad de elementos seleccionados (k), la aceleración sobre el barrido secuencial supera **9,300x**.
- 2. Desacoplamiento y Poda Selectiva de E/S (2,146 / 10,000):** En una consulta por notas en [4.00, 4.50] sobre 10,000 reportes, el índice en RAM determina que solo 2,146 cumplen la condición. El almacenamiento secundario ejecuta MultiGet únicamente sobre ellos, **evitando consultar 7,854 reportes que no pertenecen al resultado** sobre el almacenamiento persistente.
- 3. Aceleración con Filtros AMQ Modernos (Ribbon vs. Bloom):** En las escalas de 1,000 y 10,000 reportes se observó una reducción aproximada del **18.2%** frente al baseline sin filtro; con 50,000 reportes y caché caliente la ventaja desapareció (~2.3 μ s/op), evidenciando que el beneficio de los filtros depende del patrón de acceso y del estado de la caché del SO. La implementación personalizada de **Ribbon Filter** mediante la interfaz extensible *FilterPolicy* de LevelDB igualó la velocidad de Bloom (2.675 μ s vs 2.687 μ s) con un 30% menos sobrecosto de bits teórico.
- 4. Poda Temprana en Consultas Compuestas:** Ordenar los predicados por cardinalidad creciente minimiza el tamaño de los conjuntos intermedios desde el primer paso, permitiendo abortar la evaluación inmediatamente si la intersección se vacía.

6. Análisis Crítico: Debilidades y Desafíos Arquitectónicos

A pesar de sus ventajas algorítmicas, se identificaron cuatro limitaciones operativas fundamentales:

1. Concurrencia y Replicación de Índices en Memoria:

Aunque la complejidad individual de las consultas permanece baja, múltiples consultas e inserciones concurrentes en un servidor requieren políticas de sincronización sobre los árboles AVL (cerrojos de lectura/escritura), lo que induce contención bajo alta concurrencia. En una arquitectura distribuida, además, cada réplica debe mantener una versión consistente de los índices secundarios en RAM, por lo que la escalabilidad horizontal introduce costos significativos de coordinación y replicación.

2. Sobrecarga de Memoria Principal (RAM):

Mantener cuatro índices residentes en memoria implica almacenar $O(M \cdot N)$ referencias y punteros de nodos. Para cientos de miles de informes, la huella en RAM puede limitar entornos con recursos acotados, requiriendo esquemas de paginación o particionamiento.

3. Costo de Rebalanceo en Ingesta Continua:

Cada informe nuevo exige actualizar los cuatro AVL. Las rotaciones y el recálculo ascendente de (C, S, SS) (~0.25 μ s/informe) imponen un límite superior a la tasa de ingesta en comparación con motores LSM *append-only* puros.

4. Influencia de la Discretización de Claves:

El excelente comportamiento temporal observado no depende únicamente del balance AVL, sino también de la baja cardinalidad del dominio discretizado (con redondeo a 2 decimales, $D_m \leq 401$ claves distintas, $h \leq 11$, agrupando múltiples reportes en las listas L_x de cada nodo). Si se utilizara punto flotante de 64 bits sin discretizar, $D_m \approx N$, aumentando la altura del árbol, la cantidad de nodos independientes y la fragmentación en el heap.

7. Comparación con un Esquema Base sin Indexación Secundaria

El siguiente cuadro contrasta las características operativas de un esquema base sin indexación secundaria (almacenamiento plano y barrido lineal) frente a la arquitectura propuesta con índices aumentados y almacenamiento LSM:

Dimensión de Análisis	Esquema Base sin Índices Secundarios	Propuesta con Estructura Aumentada + LSM
Almacenamiento	Pasivo: Informes almacenados en archivos planos o tablas relacionales sin índices secundarios multidimensionales.	Desacoplado y Activo: Índices secundarios en RAM sincronizados con LevelDB LSM persistente.
Estadísticas de Cohorte	Fuerza Bruta $O(N)$: Escanea toda la colección para calcular media y varianza.	Aumentación $O(\log D_m)$: Cálculo en decenas de nanosegundos mediante resta de prefijos sin recorrer datos.
Selección Top-K	Fuerza Bruta con min-heap $O(N \log k)$: Costoso al acumular grandes volúmenes N .	Poda en AVL $O(\log D_m + k)$: 0.13 μ s en microbenchmark (440x más rápido que Fuerza Bruta para k=10).
Consultas Multi-Criterio	Barrido secuencial multi-filtro con degradación lineal proporcional a N .	Intersección por cardinalidad creciente con salida temprana.
Escalabilidad	Acotado a procesamiento en lotes pequeños por aula académica.	Escala de Campus: Diseñado para analítica en tiempo real sobre decenas de miles de reportes.
Eficiencia de E/S	Sin filtros probabilísticos: lecturas innecesarias para descartar reportes no coincidentes.	Filtros Ribbon y Bloom en LevelDB: hasta 18% menos latencia en hits y descarte casi instantáneo en misses.

8. Conclusiones

- 1. Validación Empírica:** Las mediciones de microbenchmark muestran tiempos del orden de decenas de nanosegundos consistentes con las cotas $O(\log D_m)$ deducidas formalmente: la aumentación transforma cálculos agregados en operaciones de costo logarítmico en el peor caso (aceleración de hasta **9,300x** frente al barrido lineal).
- 2. Viabilidad Analítica:** Los resultados indican que la arquitectura propuesta es adecuada como base para consultas analíticas interactivas sobre colecciones de decenas de miles de reportes, aunque su comportamiento bajo concurrencia pesada y despliegues distribuidos requiere evaluación adicional.
- 3. Balance Arquitectónico:** El beneficio de la aceleración analítica debe sopesarse con el consumo de memoria RAM para los índices residentes, la sincronización requerida en entornos multi-hilo y la sensibilidad a la discretización de las métricas numéricas.

Referencias Bibliográficas

[1] X. Ochoa et al., "The RAP system: Automatic feedback of oral presentation skills using multimodal analysis and low-cost sensors," in *Proc. LAK '18*. ACM, 2018.

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. MIT Press, 2022.

[3] G. M. Adelson-Velsky and E. M. Landis, "An algorithm for the organization of information," *Soviet Math. Doklady*, 1962.

[4] P. C. Dillinger and S. Walzer, "Ribbon filter: Practically smaller than bloom and xor," *CoRR*, vol. abs/2103.02515, 2021.

[5] P. E. O'Neil, E. Cheng, D. Gawlick, and E. J. O'Neil, "The log-structured merge-tree (LSM-tree)," *Acta Informatica*, 1996.